Cloud Security in AWS

The Most Common Issues

EBOOK

PGS SOFTWARE

Security is arguably the most important issue when it comes to the Cloud. It's one of the biggest concerns companies have before migrating, and it's something even Cloud-native companies check up on regularly.

In short, it's important to always be up to date with modern security standards. One way to do this is to periodically review your architecture and ensure there are no hidden exploits or issues. To help in this, here are some of the most common AWS security issues and threats we've found.

Is The Cloud Safe?

However, it's worth pointing out that, as a whole, the Cloud is safe. The big platform leaders, namely Amazon Web Services (AWS), Google Cloud Platform (GCP) and Microsoft Azure, all make great efforts to stay secure and meet various levels of certification. The problem often lies in the components and solutions built within.

For example, a recent report suggests that, between February 2018 and June 2019, 90% of all Cloud-based security issues were due to misconfiguration. There's always a story in the news, every so often, about a big company having a data leak or disclosing a breach of privacy. While these may occur on the Cloud, the root cause is nearly always a case of human error on the company's configuration side.

Note: Before we begin, I want to explain that we're talking primarily about AWS security issues here, but many of these apply to any Cloud platform, be it Azure or Google Cloud Platform. The technologies and individual solutions may change, but the overall principles do not!

Plaintext Credentials in Environmental Variables = Easy Access With Debug

If your application has debug active and encountered an error that prints a stack trace, as well as environment variables, on the screen, then your security is compromised. Start changing your passwords, rotate your keys and invalidate sessions.

Why? Because such information can be shown during these errors thanks to the debug mode, which gives unauthorised people the potential to access what they shouldn't be able to access. All it takes is a second – this is all external scanners need to copy your information and store it elsewhere. If your secrets got indexed in this way and you didn't make any changes, such people could have access for months.

Also, as we'll cover later, you should never use production credentials or accounts on non-production environments – especially those with debug enabled.

Real-world examples: This type of attack has happened to some rather big companies, most notably Gemalto and Tesla. For the latter, access credentials were gained through a very similar exploit to that described above, which then allowed for access into a storage bucket with sensitive data.

How to avoid this? Start by hiding your non-production environments behind a VPN to limit access. You can also keep your credentials in a vault service – such as AWS Parameter Store, AWS Secrets Manager or HashiCorp Vault – and extract them dynamically. This way, they aren't left open and exposed.

Keep Private S3 Buckets Private

Since we already mentioned S3 buckets, it's always important to ensure your various buckets are private and public as needed. This one might seem simple, but it's important. Public access should be very limited – anything you want to stay secure and private should be set as such.

How to avoid this? Simply enough, this involves ensuring every bucket is configured correctly. However, this is a problem of scale more than anything else – the more buckets and larger services you have, the more work it takes to check and ensure each is properly assigned.

Further, using infrastructure as code (IaC) patterns can help reduce the risk of accidentally creating a public bucket, further removing such issues in the future.

Don't Give More Permissions Than Necessary!

Giving wider permissions is a common issue in many companies – often because it's easier to configure and ensures everyone can access what they need – but it brings a whole host of risks with it.

With unregulated access in one area, users can quickly gain access elsewhere, make changes where they are not supposed to – or even welcome to – and gain access to information and processes elsewhere in the system.

Here are a few examples:

#1	Your QA team needs permission to start and stop existing instances, but they are given full access to the EC2. While the team doesn't directly have access to IAM, there is an existing role with such admin permissions. **The problem?** Your QA team can start a new instance and attach the admin role to it. This way, your QA team can now perform any action on your account using this administrator role – and not just those the QA team was originally assigned to.
#2	Your service uploads reports to a single S3 bucket and there is a policy in place to allow uploads to this specific bucket only. Your team isn't sure which permissions may be necessary here, so they set full access. **The problem?** Using this particular service, a potential attacker now has access to other buckets from this location, as their permissions aren't closed off.
#3	While setting up a new account, you decide to give all your team members admin access "in case they need it". After some time – and with no problems – you soon forget that the rest of the team also has admin access. **The problem?** A malicious insider can choose to extract sensitive data at any point, disrupting any resources you are running or even remove access for others.

As you can see, it's very easy to accidentally get into any of these situations.

How to avoid this? If you have a lot of people, it can be difficult to assign and regulate so many roles and their respective permissions. In fact, this is where a lot of the problems start, as companies give people blanket-wide permissions to save time. Instead, consider using groups and predefined roles with specific permissions for different teams and users.

You should also ensure that your general policy is set to "Default Deny". This way, rather than forgetting to remove privileges from each user, it's a case of adding the specific permissions required.

Outdated Software & Missed Security Patches

Now, this is an issue that has been around long before the Cloud – it's held true for on-site software, networks and essentially any digital solution. So, it shouldn't come as a surprise to learn this is one of the most basic Cloud security risk management practices – yet also one of the most overlooked.

Of course, we mentioned earlier that the Cloud itself is very secure. Yet, while AWS certainly patches its own managed services and hosted operating systems, it's up to individual clients regarding anything running in the virtual machines.

Ignoring updates is a fool's gambit: these updates are created as possible exploits are found. These often become public knowledge and, if you're not running the updates, it makes your service a target for malicious activity.

Here are two scenarios:

#1	You have an EC2 with permissions to access your S3 bucket(s). One of your apps running on this EC2 is outdated and is open to an exploit that allows attackers to hack into your machine. When they succeed, they will have the same permissions as your instance and can easily access your private S3 objects.
#2	You have an enterprise solution for managing tasks for multiple projects, but it is outdated. Hackers can use a well-known exploit to use it as a proxy and open any URL they wish. Because the machine runs on AWS, attackers can access its metadata and ask for a "magic" internal address which, long story short, causes a whole host of problems for you and your business.

(*No, I'm not going to tell you of any actual exploits*!)

How to avoid this? This is another simple fix – update everything on the Cloud. Likewise, ensure your IT team is always up to date with all official patches and updates. For example, the latest AWS security patches and issues are often updated regularly. Their bulletin page covers the latest common vulnerabilities and exposures (CVE) issues – so make sure your IT team is checking this on a daily basis!

Similarly, if you utilise managed services, these can automate patching, including a service window that ensures such updates don't come at the wrong time.

Never Use Shared Keys!

Shared keys are a nightmare for investigations. When you know more than one person has access to a certain key, it's no easy talk to determine responsibility. Singular keys for each individual, on the other hand, is much easier.

It's also safer, too. Shared keys often result in former team members retaining access to your machine, even when they've left the company, which is a far cry from secure. Shared keys are also much easier to leak, as people have to give them new team members etc. A singular key has no reason to be shared so often. It's not sent out via email, or written on a piece of paper!

How to solve this? Never use shared keys! Always use keys that are assigned to particular individuals. Furthermore, if you are using shared keys, disable them and assign individual access right now.

Yes, right now!
But while we're talking about keys...

Rotate Your Keys Regularly!

Even if you have individual keys, they still need to be rotated on a regular basis. A key can still leak, so the longer your keys are active, the more risk you are building up.

Yet a key rotation also helps encourage your teams to think differently. Rather than hardcoding their own access keys in various applications, they will be using less permanent, rotating solutions that are harder to crack. Here's one example to explain why:

A Junior developer is working on a lambda function and pushed his or her changes, including their access keys, to the repository. While this leak was quickly identified and removed, the change history may still be available in git. A potential attacker can extract the git history and find this key. If it's not rotated out and is still active, they then have direct access to your resources.

Real-world Examples: This is something even big tech giants can fall for. Last year, a security company was able to use leaked credentials to get inside Nokia's infrastructure. From here, they were able to find user and admin passwords, encryption keys and various private keys – everything a malicious hacker would need to cause a lot of damage. Fortunately, the company in question was testing the security and notified Nokia of the changes. But not everyone is so lucky.

Use Roles Instead Of Access Keys

In both of the last two issues, the keys themselves were the problem. When they exist as plaintext access keys, they can be easily copied and acquired. So, what if we removed this risk entirely?

Define roles, when paired with temporary and short-lived credentials from AWS Security Token Service (STS), serve the same function but are significantly safer. Similar to rotating keys, the temporary nature of STS ensures that old logs or records can't be turned into potential exploits.

In short, by removing defined keys entirely, we can remove one more data leakage scenario from the equation entirely.

Use Billing Alerts

This one is less about preventing threats, but ensuring you aren't caught unawares during such incidents. Billing Alerts are what Cloud providers like AWS use to help you stay notified on key developments. If someone gained access and redistributed your resource usage, this is what will help you know something is wrong.

Here's a clear example:

> Your current infrastructure setup – which is very stable – typically costs you $300 by the 10th of each month, $1,200 by the 20th and the final monthly bill usually costs around $2,000. Now, what happens if your keys were leaked and an attacker decided to run more costly EC2 instances (such as bitcoin mining) at your expense?

If you set up billing alerts with the figures above ($300, $1,200 and $2,000), you will get notified as soon as costs reach $300. You will then see that it's much earlier than usual, prompting you to investigate and stop these processes.

If you don't have billing alerts, the instances run unnoticed and you end up having to pay the costs. You don't notice until the end of the month, when costs are way above $2,000.

Use MFA!

Multi-factor authentication (MFA) is common in today's world. Most applications use it, including numerous banks, so why wouldn't you use it for your Cloud.

Singular-factor authentication is a clear risk – it's part of the reason why keys are so problematic. By using MFA on all of your AWS Identity and Access Management (IAM) policies, for every API call, you can keep your resources safe, even if an access key gets leaked. Without every piece, nobody is getting in without approved access.

Remove Root User Access Keys

... and let's talk about keys again! This time, let's discuss the issue with root user access keys. Because root users have access to the entire account, including resources and data, it should absolutely not.

Instead, use IAM users. Even with admin permissions, these can not take account management actions. So, even if someone did gain access to such an account, the extent of any possible damage is much more limited.

Use Separate Accounts for Production and Development Environments

Typically, development and production environments are kept clearly separate but, in the Cloud, they often share the same accounts out of ease. Similar to shared keys and unnecessary privileges, this introduces a whole host of problems. Just consider any of these less than ideal scenarios:

- The development team accidentally terminates the production machine, essentially deleting your business's service
- Production logs are accessible by unauthorised developers, who now have information they shouldn't have been able to access.

- Because everything runs on a single account, there's no clear information regarding costs – you can't easily tell if a cost spike is coming from production or non-production, for example.
- A simple, singular leak allows attackers to access your account, which now gives them access to both environments, enabling them to cause as much damage as possible.
- A developer creates an Amazon Machine Image (AMI) from the production machine and runs a development-only machine from it. While it works for the developers, this now carries all the data & configurations from production.
- A developer creates a snapshot of the production environment's Elastic Block Storage (EBS) volumes and attaches it to their development machine, giving them access to production data and configurations.

Fortunately, this is another problem with a simple answer: keep your production and non-production environment separate! You can still collect logs from any and all accounts in a single bucket, accessed via an external account for those that need it. Yet, this way, production and development do not overlap.

Block Public Access To Services Like Elasticsearch, Jenkins etc...

Management tools and other managed services, such as Jenkins or ElasticSearch, can make the Cloud even easier to configure but it's vital that the access to these tools is kept secure – in part because they often rely on standard ports. This is why we recommend that you limit this exclusively to your office(s) IP address only.

Here are two possible scenarios to consider:

#1	For example, let's assume that some of your team members want to work from home and they give you their IP address for access. While this might seem good at first, a singular IP address could be used by half of London – all of which would now have potential access if they decided to look into it.
#2	Your team members often work from different locations and, since you haven't invested in a Virtual Private Network (VPN), you keep Jenkins publicly accessible, so your team always have access. However, this is also true for anyone else and, since Jenkins is able to assume an administrative role on AWS, you have just publicly exposed a potential gateway to your whole AWS account.

Real-world Examples: This is another one that is quite simple, but nonetheless keeps popping up now and then. In the past, this has included big names and services such as Snapchat!

So, how do you know if you're safe?

The simple truth is that, unless you take the time to look, you don't know if you're safe. If you built your solutions carefully, avoided key exploits and left no backdoors available, then you have kept yourself safe. The real question is – have you tested your infrastructure to prove this?

Summary

I hope this shows that some of the biggest AWS security breaches stem from misconfiguration and improper access practices. What might seem like a simple change can in fact leave you open to much larger exploits and consequences. In fact, this is one of the first things we look at in any Cloud security risk management project – we always check for these issues before moving to more advanced issues.

It's also part of the AWS Well Architected Framework. The AWS Security pillar is a topic unto itself, but needless to say it's one of the most essential aspects of any Cloud infrastructure. The Cloud, as it stands, is no more dangerous than an on-premise server. In fact, it's usually safer since Cloud providers like AWS have clients and requirements that often exceed your own, and undertake due precaution in this regard to keep you in safe hands.

As these common security risks show, its typically poor security standards and outdated access practices that lead to the biggest risks and security threats.

Our advice? Review your current set-up and consider any and all issues.
Here's a quick checklist, just to recap:

- Are you using plaintext credentials in environmental variables with debug? **Correct Answer:** No!
- Are your private S3 buckets truly private? **Correct answer:** Yes! Our public and private buckets are configured correctly.
- Do people have more permissions than they need. **Correct answer:** No, permissions are very strict and defined.
- Have you missed some software and security patches? **Correct answer:** No, we're up to date and never miss a patch!
- Are you using shared keys? **Correct answer:** No, everyone has a singular key for their own access purposes.
- What about root user? **Correct answer:** No, we never use the root user – we have separate groups of users for all our needs and requirements.
- Are you rotating your keys? **Correct answer:** Yes! We rotate keys frequently, with unique keys for each individual.
- Do you use roles? **Correct answer:** Yes! We use roles where possible, to avoid relying on plaintext access keys.
- Do you have billing alerts set up? **Correct answer:** Yes – I want to know the moment costs spike and increase unexpectedly.
- Do you use multi-factor authentication? **Correct answer:** Of course – we use it for all access methods, including AWS IAM.
- Do you keep production and development environments apart? **Correct answer:** Always – we have seperate access to each environment. Our developers can't access production, and vice versa.
- Do you restrict access to your management tools? **Correct answer:** Yes, we only allow our office IP addresses to gain such access. (Remote works can't gain access unless its via our VPN solution.)

Business Perspective

Most AWS security issues happen on the respective customer/company's side, not because of the Cloud provider directly. Often, this is because of small errors or deviations away from modern Cloud security standards. All potential hackers need is for your company to be loose with access privileges or not careful about their online configurations. The above examples are just some of the most common AWS security threats we've experienced, but they all risk big consequences if exploited. An overall review and assessment of your Cloud architecture is always useful, but periodic review of your Cloud security is essential!

About Us

Michal Brygidyn
Cloud Solution Architect | AWS Ambassador | Security Researcher

Michal is an experienced Security Researcher (White Hat) and Cloud Solutions Architect with DevOps skillset. He is passionate about finding sources of leaking data and has already helped companies from various industries to improve their infrastructure security. His experience gained at PGS Software also covers building a successful AWS partnership including preparation for competencies and programs.

PGS Software delivers custom-made IT solutions to clients around the globe. We specialize in providing a range of services in Cloud Development, Data Science, and UX/UI Design. Our offer also extends to include Software Development, Business Analysis, and Quality Assurance. Among our existing clients are Yell, Ringier Axel Springer Media, Volotea, and many more.

Sources

- Report finds 34M vulnterabilities across AWS, Google Cloud and Azure
- Your AWS keys will be leaked or stolen. Prepare yourself
- Lessons from the Cryptojacking Attack at Tesla
- Hackerone report – AWS bucket leading to iOS test build code and configuration exposure (Slask)
- The Capital One hack couldn't have come at a worse time for Amazon's most profitable business
- Nokia exposes passwords & secret access keys to its internal systems
- Hackerone report – Open prod Jenkins instance (Snapchat)